Lean Mapping Architecture

Comprehensive Technical Reference

Serena Integration

Architecture Team

May 14, 2026

Contents

1 Executive Summary **4**
- 1.1 Purpose . . . 4
- 1.2 Business Goals . . . 4
- 1.3 Key Decisions . . . 4
- 1.4 Phased Rollout . . . 4

2 System Architecture **6**
- 2.1 Component Diagram . . . 6
- 2.2 Service Boundaries . . . 6
- 2.3 Data Flows . . . 6

3 Technology Stack **8**
- 3.1 Core Stack . . . 8
- 3.2 Key Trade-offs . . . 8
- 3.3 Optional Components . . . 8

4 API Specification **9**
- 4.1 Endpoint Catalogue . . . 9
- 4.2 Response Envelope . . . 9
- 4.3 Common Status Codes . . . 9

5 Data Model **11**
- 5.1 Entity–Relationship Overview . . . 11
- 5.2 Core Tables . . . 11
- 5.3 Indexing Strategy . . . 11
- 5.4 Materialised Views . . . 11
- 5.5 Migration Strategy . . . 12

6 Data Freshness Model **13**
- 6.1 Design Principle . . . 13
- 6.2 Cache-Control Strategy . . . 13
- 6.3 HTTP Cache Headers . . . 13
- 6.4 SSE Invalidation . . . 13
- 6.5 Staleness Alerting . . . 13
- 6.6 Consistency Guarantees . . . 14

7 Frontend Architecture **15**
- 7.1 Technology Choice . . . 15
- 7.2 Directory Layout . . . 15
- 7.3 React Query Integration . . . 15
- 7.4 Error Handling . . . 15

8 Frontend UI Patterns **16**
8.1 Three-Layer Exploration . . . 16
8.2 Graph Interaction . . . 16
8.3 Accessibility . . . 16
8.4 Export . . . 16
8.5 Performance Budget . . . 16

9 Backend Services **17**
9.1 Service Decomposition . . . 17
9.2 Full Scan Pipeline . . . 17
9.3 File Watcher . . . 17
9.4 Wolfram Service *[Optional – Non-Core]* . . . 17

10 Backend Performance **18**
10.1 Connection Pooling . . . 18
10.2 Query Optimisation . . . 18
10.3 Caching Tiers . . . 18
10.4 Performance Targets . . . 18

11 Security Model **19**
11.1 Authentication Context . . . 19
11.2 Rate Limiting . . . 19
11.3 Input Validation . . . 19
11.4 SQL Injection Prevention . . . 19
11.5 Secrets Management . . . 19
11.6 Network Segmentation . . . 19

12 Resilience Patterns **20**
12.1 Retry Policy . . . 20
12.2 Circuit Breaker . . . 20
12.3 Back-Pressure . . . 20
12.4 Graceful Degradation . . . 20
12.5 Timeout Hierarchy . . . 20

13 Observability **21**
13.1 Three Pillars . . . 21
13.2 Correlation IDs . . . 21
13.3 Key Metrics . . . 21
13.4 Alerting . . . 21
13.5 Dashboards . . . 21

14 Deployment **22**
14.1 Container Images . . . 22
14.2 Kubernetes Resources . . . 22
14.3 Zero-Downtime Deploy . . . 22
14.4 Backup . . . 22

15 Development Workflow **23**
15.1 Local Setup . . . 23
15.2 Branching . . . 23
15.3 CI Pipeline (GitHub Actions) . . . 23
15.4 Quality Gates . . . 23

16 Operational Runbook **24**
16.1 Trigger Full Reindex . . . 24
16.2 Incident Playbooks . . . 24
16.2.1 High Query Latency (p95 > 500 ms) . . . 24
16.2.2 Index Staleness (> 48 h) . . . 24
16.2.3 Redis Unavailable . . . 24
16.2.4 DB Connection Exhaustion . . . 24
16.3 Backup & Restore . . . 24

17 Decision Records **25**
17.1 ADR-001: PostgreSQL as Primary Store . . . 25
17.2 ADR-002: Redis as Ephemeral Cache . . . 25
17.3 ADR-003: Conservative Cache Invalidation . . . 25
17.4 ADR-004: SSE for Real-Time Invalidation . . . 25
17.5 ADR-005: JWT Auth for Admin Only . . . 25
17.6 ADR-006: Correlation IDs . . . 25
17.7 ADR-007: Circuit Breaker for Externals . . . 26
17.8 ADR-008: Wolfram Optional . . . 26

18 Glossary **27**

Chapter 1

Executive Summary

1.1 Purpose

The Lean Mapping Architecture provides a production-grade system for indexing, querying, and visualising dependency relationships across Lean 4 codebases (including Mathlib). It integrates with Serena as a specialised data layer.

1.2 Business Goals

Goal	Metric
Index a 10 K-file Lean project	< 5 min full scan
Symbol lookup latency (p95)	< 50 ms
Incremental file update	< 2 s end-to-end
Transitive dependency query (depth 5)	< 200 ms
System availability	99.9 % monthly

1.3 Key Decisions

1. PostgreSQL 16+ as primary store (JSONB, recursive CTEs, FTS).
2. Redis 7+ as ephemeral cache—never the source of truth.
3. Three stateless FastAPI services (Query, Ingestion, Admin).
4. Next.js 15 frontend with React Flow graph visualisation.
5. Wolfram integration is optional and non-core.

1.4 Phased Rollout

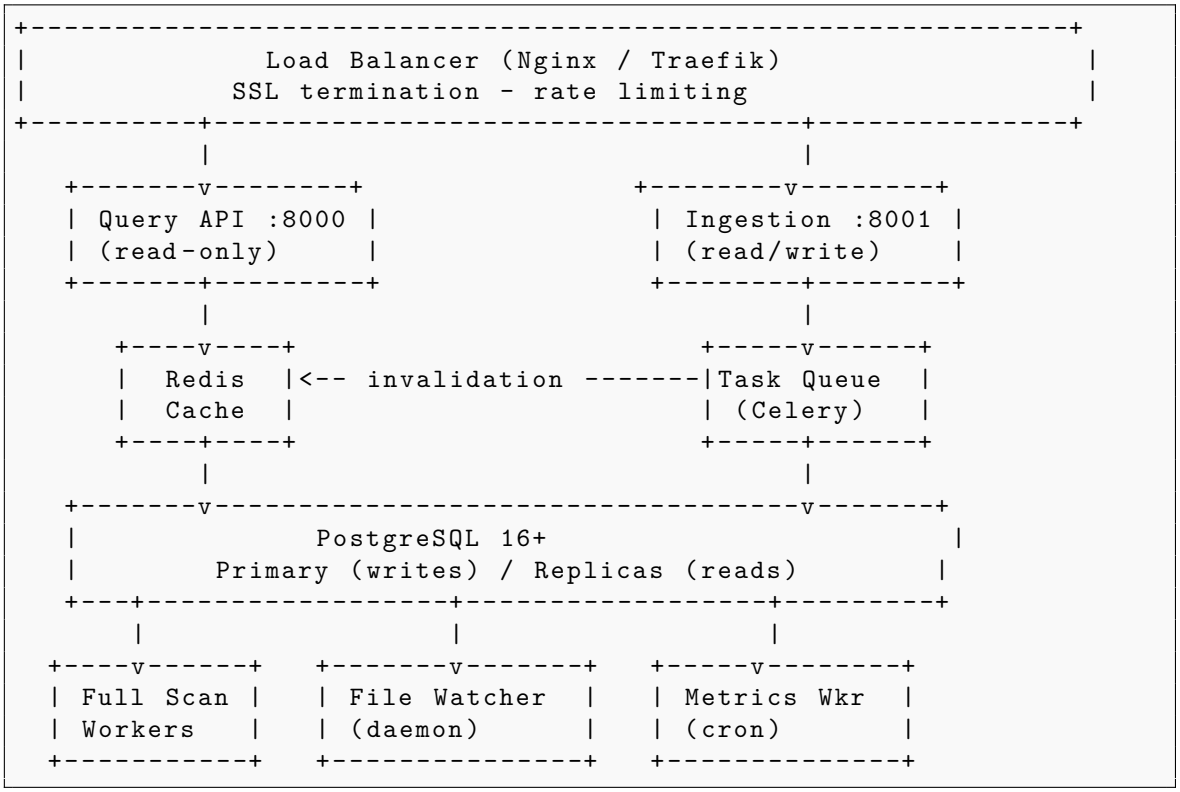
Phase	**Weeks**	**Deliverable**
Foundation	1–3	Schema, full-scan ingestion
Incremental	4–5	File watcher, Redis caching
API Layer	6–8	Query/Search/Viz endpoints
Frontend	9–10	Three-layer exploration UI
Hardening	11–12	Observability, runbooks, load tests
Optional	Post-MVP	Neo4j export, Wolfram enrichment

Chapter 2

System Architecture

2.1 Component Diagram

High-level component map



2.2 Service Boundaries

2.3 Data Flows

Read path: Client → LB → Query API → Redis (L1). On miss: PostgreSQL query, cache result, return with freshness metadata.

Incremental write: FS event → File Watcher (debounce 500 ms) → hash check → transactional update (delete old, insert new) → invalidate Redis → optionally enqueue dependent file checks.

Service	Responsibility	Scaling
Query API	Read-only lookups, search, viz	Horizontal
Ingestion API	Accept index/reindex requests	Horizontal
Admin API	Cache control, migrations	Single
Full Scan Wkr	Complete project indexing	Celery pool
File Watcher	Incremental updates on FS events	1 per project
Metrics Worker	Refresh materialised views	Cron
Wolfram Svc	*[Optional]* Math enrichment	Separate

Full scan write: Ingestion API → Celery task → five-phase pipeline (discover, imports, symbols, dependencies, finalise) → update `index_runs`.

Chapter 3

Technology Stack

3.1 Core Stack

Layer	Technology	Version	Rationale
Primary DB	PostgreSQL	16+	JSONB, recursive CTEs, FTS, ACID
Cache	Redis	7+	Sub-ms lookups, TTL, pattern delete
API framework	FastAPI	0.110+	Async, auto-OpenAPI, Pydantic v2
ASGI server	Uvicorn	0.29+	High-perf async server
Task queue	Celery + Redis	5.3+	Distributed task processing
File watcher	watchdog	4+	Cross-platform FS events
Frontend	Next.js 15	15+	RSC, streaming, SSR
Graph viz	React Flow	11+	1000+ node perf, custom nodes
Charts	Recharts	2.12+	Composable, Tailwind-friendly
State mgmt	Zustand	4+	Minimal boilerplate
Styling	Tailwind CSS	3.4+	Utility-first, dark mode

3.2 Key Trade-offs

- **PostgreSQL over MongoDB**: referential integrity for dependency graphs outweighs schema flexibility.
- **Redis as ephemeral cache only**: on total cache loss, the system degrades to DB queries—no data is lost.
- **Neo4j deferred**: recursive CTEs sufficient for depth ≤ 10; adopt Neo4j only if interactive graph exploration becomes a priority.

3.3 Optional Components

Component	Adoption trigger
Neo4j Community	Visual graph exploration priority
Wolfram Engine	Mathematical verification / LaTeX proofs
Grafana + Loki	Log aggregation beyond stdout

Chapter 4

API Specification

All endpoints use the versioned prefix `/api/v1/`.

4.1 Endpoint Catalogue

Method	Path	Auth	Description
GET	`/symbols/{name}`	–	Symbol lookup
GET	`/symbols/{name}/dependencies`	–	Forward deps
GET	`/symbols/{name}/usages`	–	Reverse deps
GET	`/search`	–	Full-text search
GET	`/files/{mod}/imports`	–	Module imports
GET	`/viz/project/{name}/graph`	–	Viz export
GET	`/charts/symbol-distribution`	–	Analytics
POST	`/admin/index/start`	JWT	Trigger index
GET	`/admin/index/status/{id}`	JWT	Job progress
GET	`/health`	–	Readiness

4.2 Response Envelope

Every successful response includes a `metadata` block:

```
{
  "metadata": {
    "indexed_at": "2025-01-15T10:30:00Z",
    "cache_hit": false,
    "freshness": { "max_age_s": 3600, "source": "database" }
  }
}
```

Error responses carry a consistent envelope with `error.code`, `error.message`, and `error.request_id` (the correlation ID).

4.3 Common Status Codes

Code	Meaning
200	Success
202	Accepted (async job)
400	Bad request / validation
401	Missing/invalid JWT
404	Not found
429	Rate limited
500	Internal error
503	Unavailable (DB down)

Chapter 5

Data Model

5.1 Entity–Relationship Overview

```
projects 1--* files 1--* symbols
                 |              |
                 |              +--* dependencies (source -> target)
                 |              +--* external_refs
                 |
                 +--* file_dependencies (source_file -> target_file)

index_runs *--1 projects
```

5.2 Core Tables

Table	Key columns	Notes
`projects`	name, root_path, lean_version	UNIQUE(name)
`files`	module_name, file_hash, imports (JSONB)	UNIQUE(project, path)
`symbols`	full_name, kind (ENUM), signature, docstring	UNIQUE(project, full_name)
`dependencies`	source, target, type (ENUM), confidence	Composite unique
`file_dependencies`	source_file, target_file	Module imports
`external_refs`	symbol_id, external_name, mathlib_url	Mathlib links
`index_runs`	run_type, status, errors (JSONB)	Audit trail

5.3 Indexing Strategy

- B-tree on `symbols(full_name)` for $O(1)$ lookup.
- GIN on `symbols(attributes)` for JSONB containment.
- GIN/tsvector on `symbols(docstring || name)` for FTS.
- Composite B-tree on `dependencies(source, target)`.

5.4 Materialised Views

dependency_stats: per-symbol dependency count, usage count, centrality score. **module_metrics**: per-module symbol count by kind, average length. Both refreshed concurrently

on an hourly schedule.

5.5 Migration Strategy

Alembic manages schema versions. Every migration is forward-only in production, tested in CI, and backwards-compatible for one release cycle.

Chapter 6

Data Freshness Model

6.1 Design Principle

Every API response includes freshness metadata so consumers can assess data currency.

6.2 Cache-Control Strategy

Resource	TTL	Invalidation trigger
Symbol metadata	1 h	File hash change
Dependency subgraph	1 h	Source/target symbol change
Search results	15 min	Any symbol mutation
Module import graph	1 h	File dependency change
Materialised views	1 h	Cron refresh

6.3 HTTP Cache Headers

Query API responses include:

```
Cache-Control: public, max-age=60, stale-while-revalidate=300
ETag: "sha256:<hash>"
```

6.4 SSE Invalidation

An optional `/api/v1/events/invalidation` SSE endpoint streams real-time invalidation events. The frontend listens and calls `queryClient.invalidateQueries` on receipt.

6.5 Staleness Alerting

- `indexed_at` $>$ 24 h: warning badge in UI.
- `indexed_at` $>$ 48 h: Prometheus alert `index_stale`.
- Redis unavailable: fallback to DB; log `cache_miss_fallback`.

6.6 Consistency Guarantees

Serialisable isolation for index writes (strong within transaction). Cache reads are eventually consistent (worst-case staleness = TTL). SSE reduces effective staleness to sub-second for connected clients.

Chapter 7

Frontend Architecture

7.1 Technology Choice

Next.js 15 (App Router) with React 19. Server Components for data-heavy views; Client Components for interactivity. State: Zustand (global) + React Query (server state). Styling: Tailwind CSS 3.4.

7.2 Directory Layout

```
frontend/
  app/                    # Next.js App Router
    projects/[name]/      # Layer 1 - Project overview
      modules/[mod]/      # Layer 2 - Module detail
        symbols/[sym]/    # Layer 3 - Symbol detail
    search/page.tsx
  components/
    visualizations/       # DependencyGraph, SymbolChart
    exploration/          # BreadcrumbNav, SymbolList
    search/               # SearchBar, FilterPanel
    ui/                   # Shared primitives
  lib/
    api/                  # Typed fetch client
    graph/                # Layout algorithms
    hooks/                # useSymbol, useDependencies
  state/                  # Zustand store + slices
```

7.3 React Query Integration

Each data type has a dedicated hook with appropriate `staleTime`: symbol metadata (5 min), dependency graphs (10 min), search results (2 min). Prefetching on hover for instant drill-down.

7.4 Error Handling

Global `ErrorBoundary` for render errors. API errors surface via React Query's `error` state—toasts for transient, inline for 404. Network errors trigger exponential back-off retry (max 3).

Chapter 8

Frontend UI Patterns

8.1 Three-Layer Exploration

1. **Project Overview** (`/projects/{name}`): module-level dependency graph + project metrics.
2. **Module Detail** (`.../modules/{mod}`): filterable symbol list + symbol-level graph.
3. **Symbol Detail** (`.../symbols/{sym}`): metadata card, dependency subgraph, code preview.

8.2 Graph Interaction

Click: navigate. Hover: tooltip. Scroll: zoom. Drag: pan. Ctrl+click: multi-select. Escape: deselect.

Custom node colours per `symbol.kind`: theorem (blue `#2563eb`), def (purple `#7c3aed`), axiom (red `#dc2626`), instance (green `#059669`). All meet WCAG 2.1 AA (≥ 4.5:1).

8.3 Accessibility

Skip links, ARIA live region for node focus, table alternative for every graph, 2 px focus indicators. Keyboard: Cmd+K (search), Cmd+B (sidebar), arrow keys (graph navigation).

8.4 Export

Cytoscape JSON, GraphML, SVG—triggered via toolbar buttons.

8.5 Performance Budget

Metric	Target
Time to Interactive	< 3 s
LCP	< 2.5 s
Graph render (1 000 nodes)	< 1 s
Bundle (gzipped, initial)	< 250 KB

Chapter 9

Backend Services

9.1 Service Decomposition

Service	Port	Role	Data access
Query API	8000	R/O lookups, search, viz	PG replica + Redis
Ingestion API	8001	Index requests, job status	PG + Celery
Admin API	8002	Cache, migrations, health	PG + Redis + queue
Full Scan Wkr	–	Complete indexing	PG (writes)
File Watcher	–	Incremental updates	PG + Redis
Metrics Wkr	–	Materialised view refresh	PG
Wolfram Svc	–	*[Optional]* enrichment	PG metadata

9.2 Full Scan Pipeline

Five phases: discover → import pass → symbol pass → dependency pass → finalise. All within a single PG transaction per batch.

9.3 File Watcher

Long-running daemon; debounce 500 ms; SHA-256 hash gate; transactional update; cache invalidation; optional dependent notification.

9.4 Wolfram Service *[Optional – Non-Core]*

Shells out to `wolframscript`; stores result in `symbols.metadata` JSONB. No other service depends on it.

Chapter 10

Backend Performance

10.1 Connection Pooling

`asyncpg` pool: `min_size=10`, `max_size=50`, `command_timeout=30 s`. For >50 concurrent replicas, introduce PgBouncer in transaction-pooling mode.

10.2 Query Optimisation

Transitive dependencies via recursive CTE with depth cap. Full-text search via GIN index on `to_tsvector`. Batch inserts via `executemany` in batches of 500 rows.

10.3 Caching Tiers

Tier	Tech	TTL	Scope
L1	Redis	1 h / 15 min	Cluster-wide
L2	HTTP headers	60 s + SWR	CDN / browser
L3	React Query	5–10 min	Per browser tab

10.4 Performance Targets

Operation	p95 target
Symbol lookup (cached)	5 ms
Symbol lookup (uncached)	50 ms
Transitive deps (depth 5)	200 ms
FTS (10 K symbols)	100 ms
Full scan (10 K files)	5 min
Incremental update (1 file)	2 s

Chapter 11

Security Model

11.1 Authentication Context

Read-only endpoints are public with IP-based rate limiting. Admin/Ingestion endpoints require an HS256 JWT bearer token validated by middleware. Signing key from `LEAN_MAP_JWT_SECRET` env var.

11.2 Rate Limiting

Tier	Limit	Scope
Anonymous	60 req/min	Per IP
Authenticated	600 req/min	Per JWT `sub`

11.3 Input Validation

All parameters validated by Pydantic v2. `full_name` regex: `^[A-Za-z_][A-Za-z0-9_.]*$`. `depth` clamped to $[1, 10]$.

11.4 SQL Injection Prevention

All queries use parameterised statements (`$1`, `$2`) via `asyncpg`. No string interpolation in any SQL path.

11.5 Secrets Management

Environment variables in Kubernetes `Secret` objects; never baked into images.

11.6 Network Segmentation

Admin API not exposed publicly. PostgreSQL and Redis listen on private subnet only. Egress restricted to Mathlib docs API and (optionally) Wolfram Cloud.

Chapter 12

Resilience Patterns

12.1 Retry Policy

Operation	Max retries	Back-off	Jitter
DB query (transient)	3	Exponential (100 ms)	±50 ms
Redis GET/SET	2	Fixed 50 ms	–
Celery task	3	Exponential (1 s)	±500 ms
Mathlib API	2	Exponential (500 ms)	±200 ms

12.2 Circuit Breaker

Wraps calls to Mathlib docs API and Wolfram Engine. States: CLOSED → OPEN (after 5 failures) → HALF-OPEN (after 30 s) → CLOSED (on probe success).

12.3 Back-Pressure

Celery queue depth > 1 000: Ingestion API returns 429. `asyncpg` pool exhaustion: API returns 503 + `Retry-After`.

12.4 Graceful Degradation

- Redis down: queries fall through to PostgreSQL.
- Celery down: ingestion queued in-memory; manual retry.
- File Watcher down: next full scan catches up.
- Wolfram down: enrichment skipped; core data unaffected.

12.5 Timeout Hierarchy

Client (30 s) > LB (25 s) > API (20 s) > DB (command_timeout 30 s but query-level 20 s). Each layer strictly shorter than its caller.

Chapter 13

Observability

13.1 Three Pillars

Metrics: Prometheus + Grafana. **Logs**: structured JSON to stdout. **Traces**: OpenTelemetry → Jaeger.

13.2 Correlation IDs

Every request carries `X-Request-ID`—attached to all log lines, propagated to Celery tasks, returned in responses and error envelopes.

13.3 Key Metrics

Metric	Type	Labels
`api_request_duration_seconds`	Histogram	method, endpoint, status
`db_query_duration_seconds`	Histogram	query_type
`cache_operations_total`	Counter	operation
`index_staleness_seconds`	Gauge	project
`celery_queue_depth`	Gauge	queue_name

13.4 Alerting

Alert	Condition	Severity
HighQueryLatency	p95 $>$ 500 ms for 5 min	Warning
CacheHitRateLow	$<$ 70 % for 15 min	Warning
IndexStale	$>$ 48 h	Critical
ErrorRateHigh	5xx $>$ 1 % for 5 min	Critical

13.5 Dashboards

Pre-built Grafana dashboards: Overview (rate, errors, latency), Cache (hit/miss, memory), Indexing (duration, errors), Database (connections, query latency).

Chapter 14

Deployment

14.1 Container Images

Image	Base	Port
lean-map-query	python:3.12-slim	8000
lean-map-ingest	python:3.12-slim	8001
lean-map-admin	python:3.12-slim	8002
lean-map-worker	python:3.12-slim	–
lean-map-watcher	python:3.12-slim	–
lean-map-frontend	node:20-slim	3000
postgres	postgres:16-alpine	5432
redis	redis:7-alpine	6379

14.2 Kubernetes Resources

Query API: 3 replicas, 250 m–1 CPU, 256–512 Mi. Workers: 4 replicas, 500 m–2 CPU, 512 Mi–1 Gi.

14.3 Zero-Downtime Deploy

Rolling update (`maxUnavailable: 0`, `maxSurge: 1`). Readiness probe: `GET /health`. Liveness: same, 30 s initial delay. Migrations run as a Kubernetes Job before deployment.

14.4 Backup

Daily `pg_dump` to object storage; WAL archiving for PITR. Redis: no backup (ephemeral).

Chapter 15

Development Workflow

15.1 Local Setup

1. Clone and `pip install -e ".[dev]"`.
2. `docker compose up -d postgres redis`.
3. `alembic upgrade head`.
4. `uvicorn app.query:app -reload -port 8000`.
5. `cd frontend && npm install && npm run dev`.

15.2 Branching

`main` (stable) ← `develop` (integration) ← `feat/<ticket>` / `fix/<ticket>`.

15.3 CI Pipeline (GitHub Actions)

On PR: lint (Ruff, mypy, ESLint, tsc), unit tests (pytest, vitest), integration tests (testcontainers), security scan (pip-audit, npm audit), migration test.

On merge to main: build images, push to registry, deploy to staging.

15.4 Quality Gates

Gate	Tool	Threshold
Python lint	Ruff	Zero errors
Python types	mypy (strict)	Zero errors
Python tests	pytest	80 % coverage
JS lint	ESLint	Zero errors
JS types	tsc –noEmit	Zero errors
JS tests	Vitest	80 % coverage
CVEs	pip-audit / npm audit	Zero high/critical

Chapter 16

Operational Runbook

16.1 Trigger Full Reindex

```
curl -X POST https://internal/api/v1/admin/index/start \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"project_path":"/data/mathlib4","mode":"full_scan"}'
```

Monitor: `GET /api/v1/admin/index/status/{run_id}`.

16.2 Incident Playbooks

16.2.1 High Query Latency (p95 $>$ 500 ms)

Check Grafana DB dashboard → `pg_stat_activity` → cancel long queries → verify Redis health.

16.2.2 Index Staleness ($>$ 48 h)

Inspect `index_runs` → check errors JSONB → restart watcher if stuck → re-trigger full scan.

16.2.3 Redis Unavailable

Auto-degrades. Verify recovery via Grafana cache-hit metric within 15 min.

16.2.4 DB Connection Exhaustion

Check `pg_stat_activity` → terminate idle-in-transaction → consider PgBouncer.

16.3 Backup & Restore

```
# Backup
pg_dump -Fc lean_map > lean_map_$(date +%Y%m%d).dump

# Restore
pg_restore --clean lean_map_20250713.dump
```

WAL archiving enabled for point-in-time recovery.

Chapter 17

Decision Records

17.1 ADR-001: PostgreSQL as Primary Store

Status: Accepted.
Decision: PostgreSQL 16+ with JSONB, recursive CTEs, and FTS. MongoDB rejected (no FK enforcement); Neo4j-only rejected (weak FTS); SQLite rejected (concurrent writes).

17.2 ADR-002: Redis as Ephemeral Cache

Status: Accepted.
Decision: Redis 7+ cache-aside, never source of truth. Graceful degradation to DB on failure.

17.3 ADR-003: Conservative Cache Invalidation

Status: Accepted.
Decision: On file change, delete file + module symbol + dependency caches. Correctness over hit rate.

17.4 ADR-004: SSE for Real-Time Invalidation

Status: Accepted.
Decision: `/events/invalidation` SSE stream. Frontend invalidates React Query keys on receipt.

17.5 ADR-005: JWT Auth for Admin Only

Status: Accepted.
Decision: HS256 JWT for write endpoints; rate limiting for reads.

17.6 ADR-006: Correlation IDs

Status: Accepted.
Decision: `X-Request-ID` on every request, in all logs, Celery metadata, and responses.

17.7 ADR-007: Circuit Breaker for Externals

Status: Accepted.
Decision: 5-failure threshold, 30 s open, half-open probe.

17.8 ADR-008: Wolfram Optional

Status: Accepted.
Decision: Separate container, no critical-path dependency.

Chapter 18

Glossary

Term	Definition
ADR	Architecture Decision Record.
Cache-aside	Pattern: check cache before DB; populate on miss.
Celery	Distributed task queue for Python.
Circuit breaker	Stop calling a failing service after N errors.
Correlation ID	Unique `X-Request-ID` per logical request.
CTE	Common Table Expression; `WITH RECURSIVE` for graph queries.
Dependency (direct)	Explicitly referenced symbol in a definition/proof.
Dependency (implicit)	Resolved via typeclass inference.
Dependency (transitive)	Reachable through a chain of direct/implicit deps.
FTS	Full-Text Search (PostgreSQL `tsvector`/`tsquery`).
GIN index	Generalised Inverted Index for JSONB and FTS.
Lean 4	Functional language and interactive theorem prover.
Materialised view	PG view stored on disk, refreshed on demand.
Mathlib	Community math library for Lean 4 ($\sim$100 K theorems).
Module	Lean namespace corresponding to a `.lean` file.
React Flow	React library for interactive node-based graphs.
React Query	Data-fetching and caching for React (TanStack).
RSC	React Server Components.
SSE	Server-Sent Events.
Symbol	Named Lean entity (theorem, def, axiom, instance, ...).
TTL	Time-To-Live for cached entries.
Wolfram	*[Optional]* Mathematical engine; non-critical path.
Zustand	Lightweight React state management.